Lecture 3.2: Advanced AWS Compute

Maharishi International University

Department of Computer Science

M.S. Thao Huy Vu

Maharishi International University - Fairfield, Iowa

All rights reserved. No part of this slide presentation may be reproduced or utilized in any form or by any means, electronic or mechanical, including photocopying or recording, or by any information storage and retrieval system, without permission in writing from Maharishi International University (MIU).

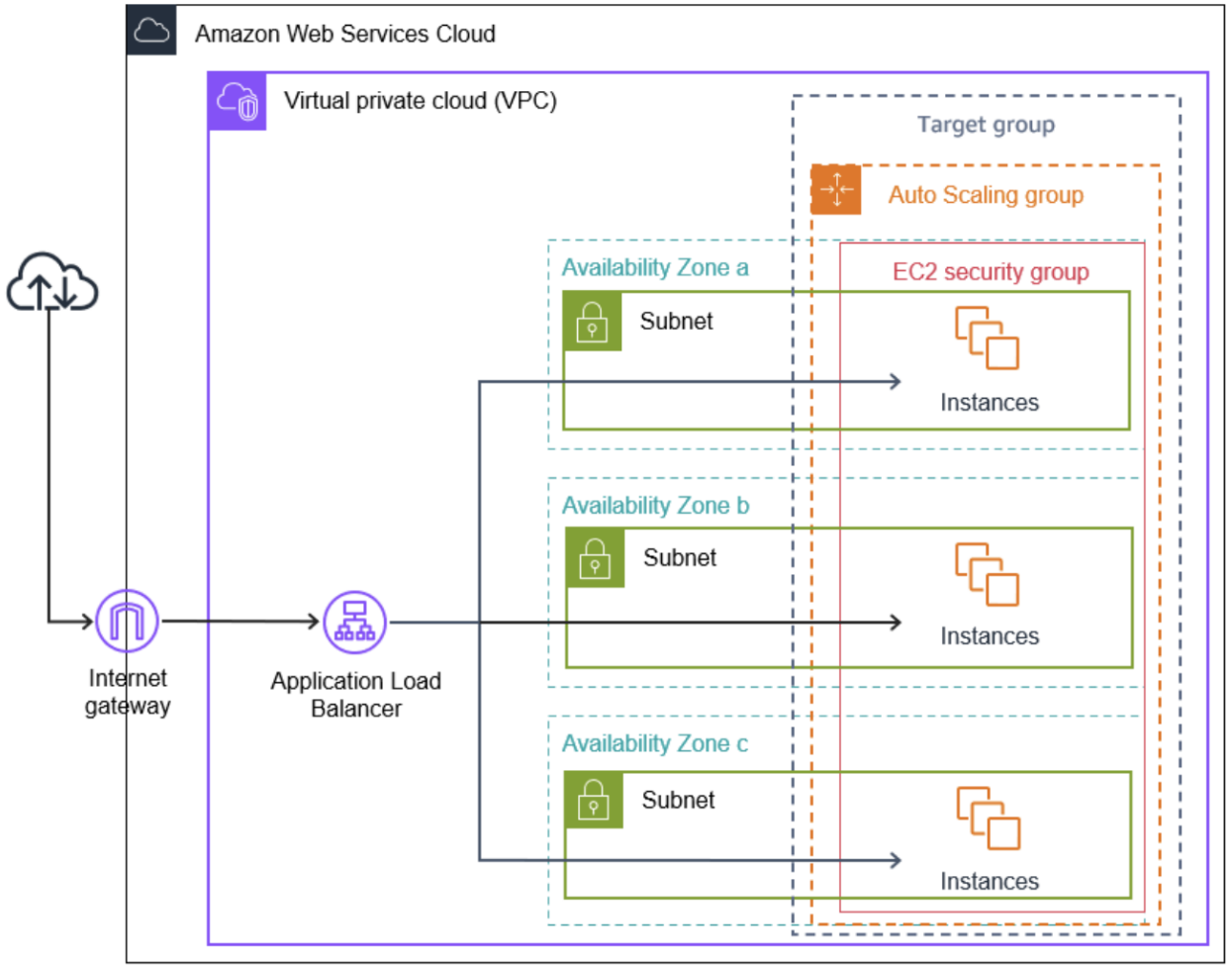
Agenda

- ASG
- Beanstalk
- AWS Lambda
- API Gateway
- Cognito

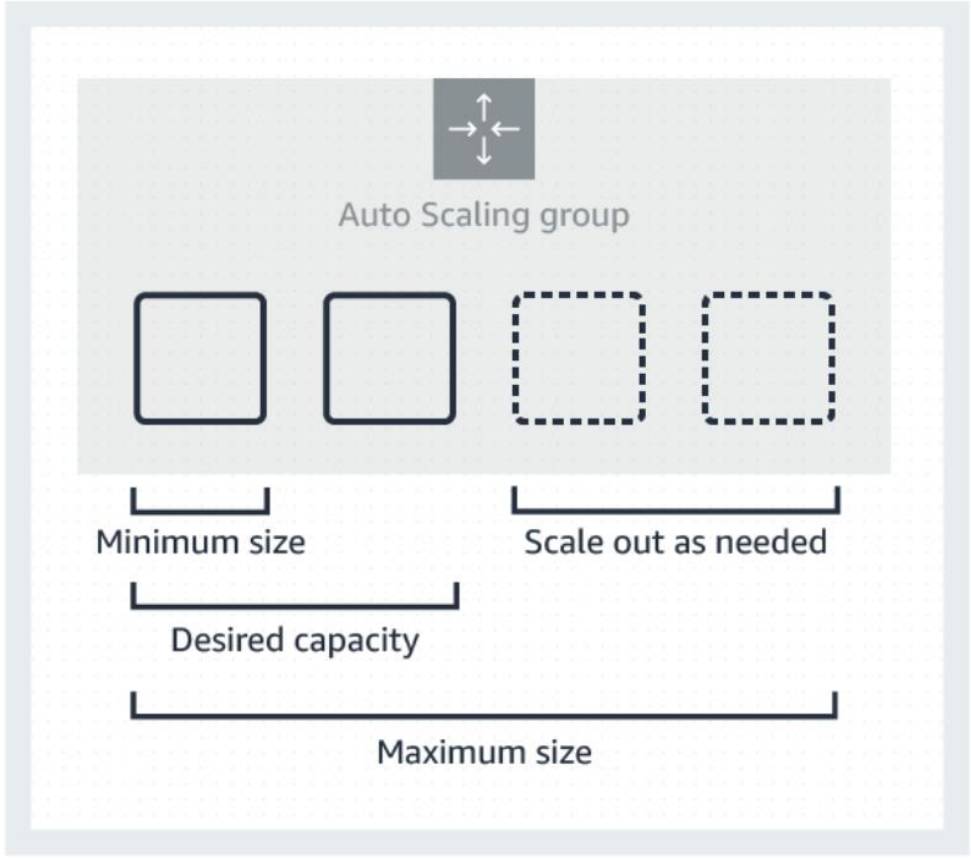
Auto Scaling Group

- Auto Scaling Group (ASG) automatically scales out or in scalable resources such as EC2, Elastic Container Service (ECS), Lambda, Aurora replicas (RDS), DynamoDB, and more.
- Scaling out: Adding more instances
- Scaling in: Removing instances
- It changes the desired number of resources running. The desired number is an ideal number of resources to meet the demand. If the current number of instances is 3 and desired number is 4, it will add 1 instance.

Auto Scaling Group



Auto Scaling Group



Auto Scaling and Load Balancing

- Auto Scaling and Load balancing: Balance traffic to multiple targets and improve application performance
- Auto Scaling: Automatically adjusts the number of instances based on application requirements, such as CPU load or SQS queue messages, to handle varying workloads
- ELB (Elastic Load Balancing): Automatically distributes incoming application traffic across multiple targets or virtual appliances in one or more AZs (Availability Zones)

Auto Scaling

- Auto Scaling Group: Maintain the health and desired count of instances
- Launch Configurations/Templates: Define the instance configuration for use in an Auto Scaling Group
- Scaling Policies: How to scale out or in

Launch template and ASG

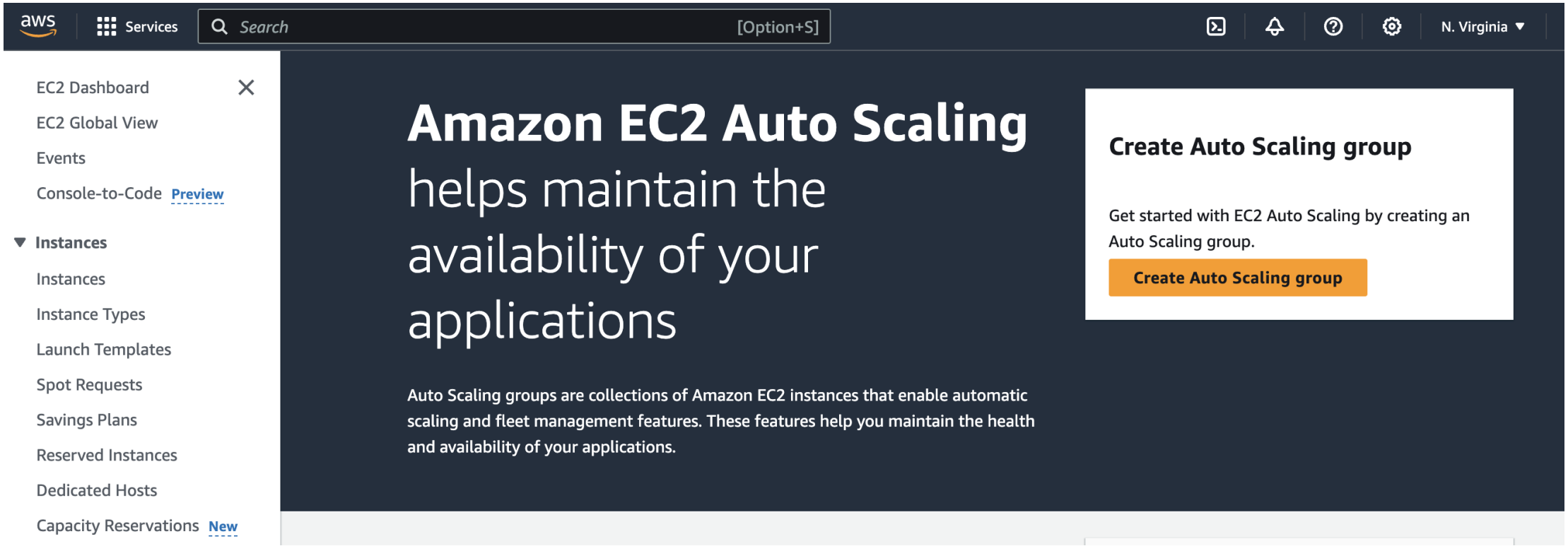
- Launch template contains
  - AMI
  - instance type
  - IAM profile
  - User Data – Startup script. It will be executed when the server starts.
  - SG and so on.
- ASG contains
  - Networking (AZs)
  - Scaling policies
  - Load Balancer
  - Health check configuration

Scaling policies

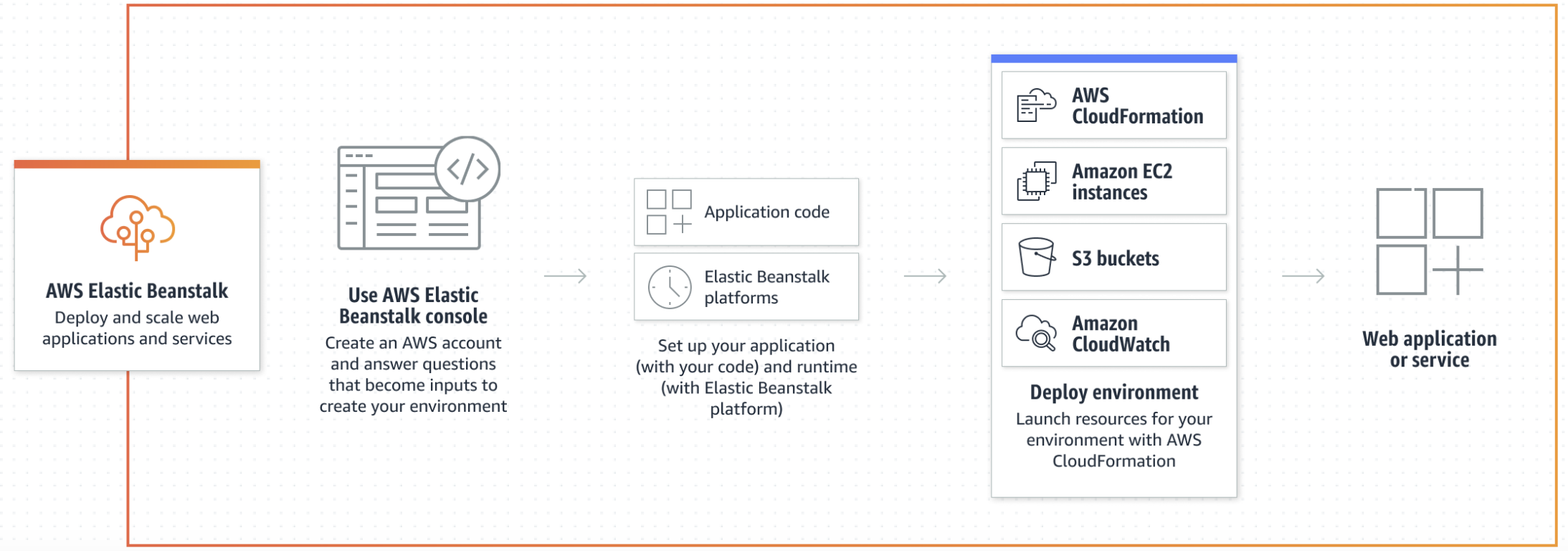
Application Auto Scaling allows you to automatically scale your scalable resources according to conditions that you define:

1. **Target tracking scaling** - Scale a resource based on a target value for a specific CloudWatch metric. Like the thermostat at your home, it maintains the metric at that threshold.
2. **Step scaling –** With step scaling, you can set complex scaling policies based on custom metrics. It allows you to set custom rules.
3. **Scheduled scaling** - Scale a resource based on the date and time. For example, more instances from 9 am to 5 pm.
4. **Predictive scaling** - Uses machine learning to analyze each resource's historical workload and regularly forecasts the future load for the next two days. Based on 2-week data, guesses the next 48 hours.

Demo


aws
Services
Search
[Option+S]
N. Virginia
EC2 Dashboard
EC2 Global View
Events
Console-to-Code Preview
Instances
Instances
Instance Types
Launch Templates
Spot Requests
Savings Plans
Reserved Instances
Dedicated Hosts
Capacity Reservations New
Amazon EC2 Auto Scaling
helps maintain the availability of your applications
Auto Scaling groups are collections of Amazon EC2 instances that enable automatic scaling and fleet management features. These features help you maintain the health and availability of your applications.
Create Auto Scaling group
Get started with EC2 Auto Scaling by creating an Auto Scaling group.
Create Auto Scaling group

Amazon Elastic Beanstalk



Amazon Elastic Beanstalk

- A service for deploying and scaling web applications and services on AWS Ecosystem
- Simplifies cloud provisioning, deployment, and scaling.

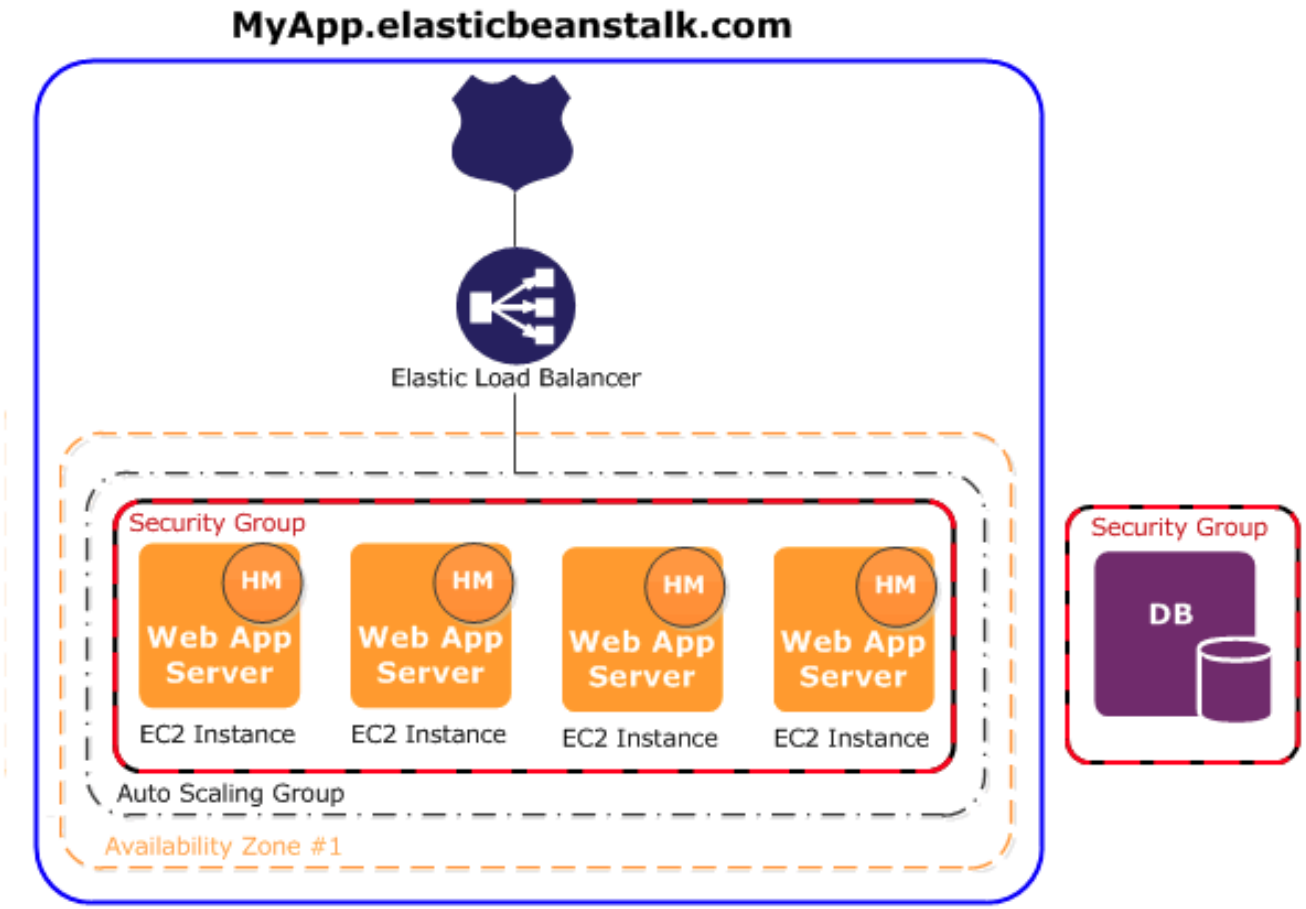
Amazon Elastic Beanstalk

- Key features:
  - Automatic Resource Provisioning: Capacity, load balancing, health monitoring
  - Developer Tools Integration: Eclipse, Visual Studio, and Git.
  - Environment Management: Auto tweak the underlying resources without needing to delve into the complexities of AWS infrastructure.
  - Multi-language Support: Support multiple programming languages and frameworks

Amazon Elastic Beanstalk

- Permissions:
  - Service role: Elastic Beanstalk assumes a service role to use other AWS services on your behalf
    - **AWSElasticBeanstalkEnhancedHealth**
    - **AWSElasticBeanstalkManagedUpdatesCustomerRolePolicy**
  - Instance Profile: Elastic Beanstalk applies instances profile to the instances in your environment.
    - **AWSElasticBeanstalkWebTier**
    - **AWSElasticBeanstalkWorkerTier**
    - **AWSElasticBeanstalkMulticontainerDocker**

Amazon Elastic Beanstalk



AWS Lambda

- AWS Lamda is a serverless computing service that allows developers to run their code without caring about servers
- Zero administrative effort required for scalability, back-ups, or multi-data center redundancy

AWS Lambda

- Features
  - Even-driven Execution: It is triggered by other services or applications
  - Automatic Scaling
  - No server management
  - Pricing based on Actual Usage

AWS Lambda

- Configuration:
  - Language: Python, NodeJS, Java, etc.
  - Execution Role and Permissions: IAM Role that a Lambda function assumes during execution to interact with other AWS services.
  - Environment variables: To manage configuration settings and secrets within Lambda functions.

What is stateful and stateless app?

Stateful applications and processes are those that can be returned to again and again, like online banking or email. They're performed with the **context of previous transactions** and the current transaction may be affected by what happened during previous transactions. For these reasons, stateful apps use **the same servers** each time they process a request from a user.

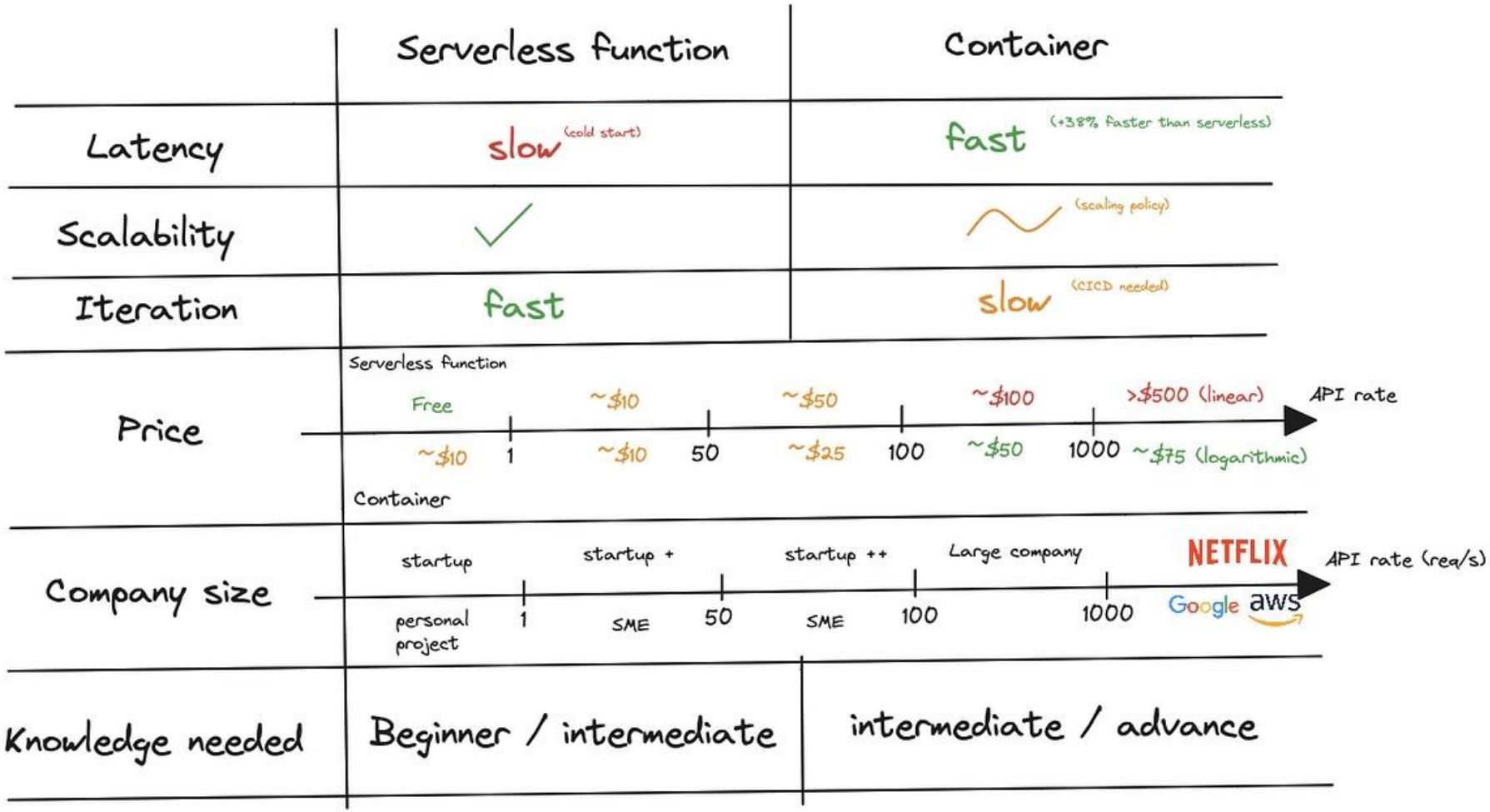
A **stateless** process or application can be understood in isolation. There is **no stored knowledge** of or reference to past transactions. Each transaction is made as if from scratch for the first time.

Keeping apps stateless allows apps to scale freely in any type of app such as EC2, ECS, Lambda. Keeping functions stateless enables AWS Lambda to rapidly launch as many copies of the function as needed to scale to the rate of incoming events. While AWS Lambda's programming model is stateless, your code can access stateful data by calling other web services, such as Amazon S3 or Amazon DynamoDB.

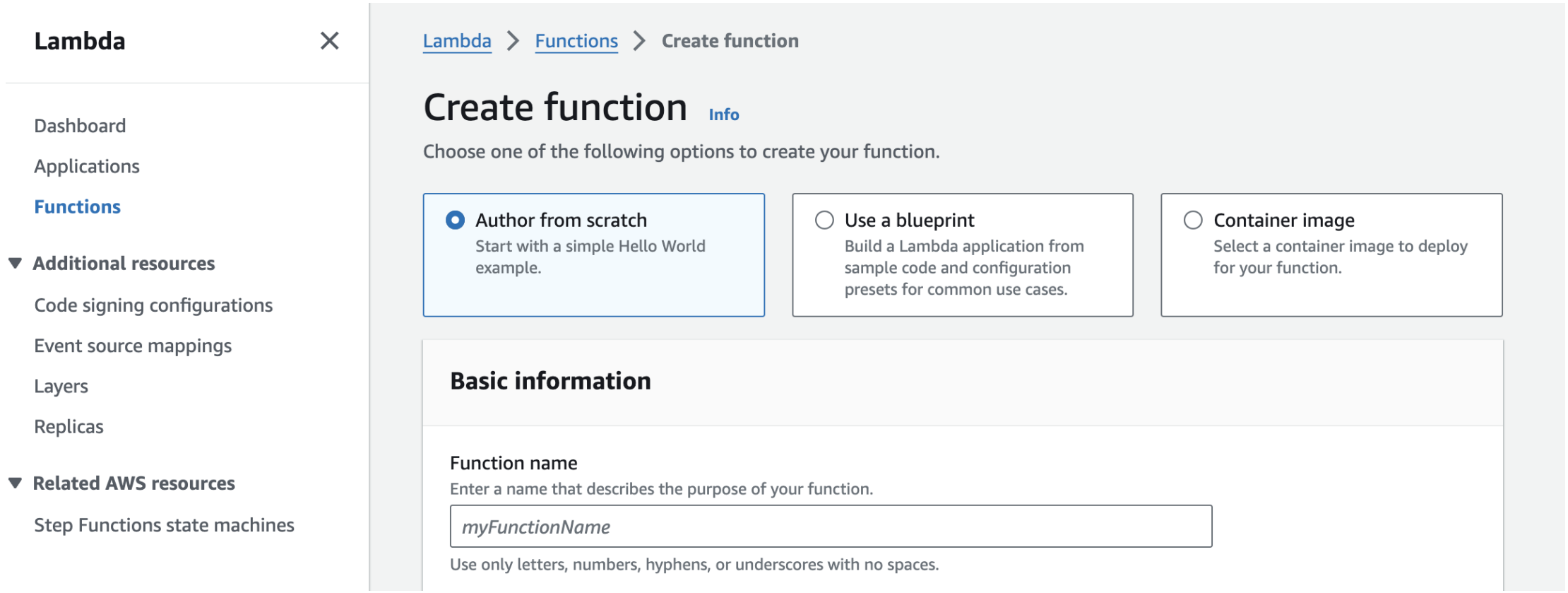
When should I use Lambda vs EC2

- Use lambda when
  - you don’t have many developers to manage servers
  - you want faster development
  - number of transactions are **unpredictable**
  - the max number of transactions are thousands at a second
- Use EC2 when
  - you need to manage the underlying resources
  - number of transactions are predictable, relatively consistent over time.
  - large number of transactions at a time
  - 24/7

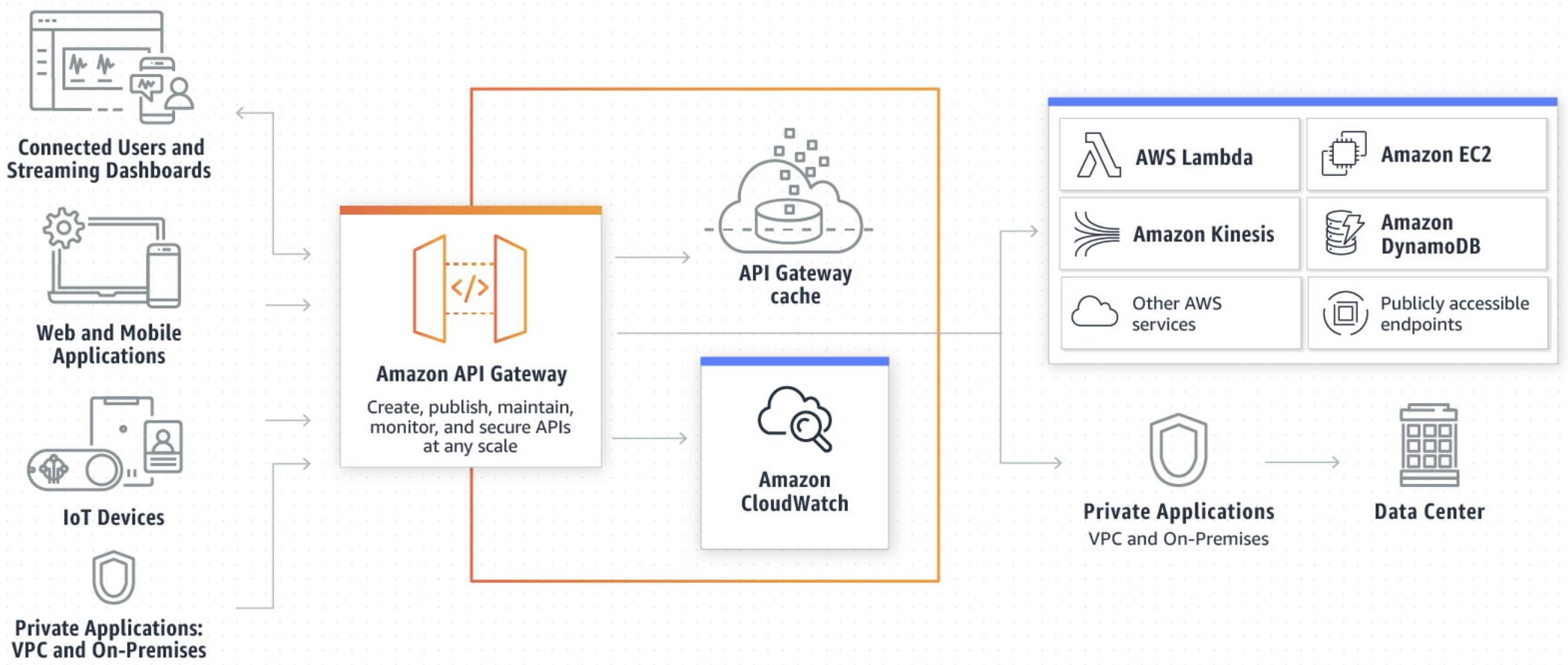
Lambda VS ECS



Demo


Lambda
Dashboard
Applications
Functions
Additional resources
Code signing configurations
Event source mappings
Layers
Replicas
Related AWS resources
Step Functions state machines
Lambda > Functions > Create function
Create function Info
Choose one of the following options to create your function.
Author from scratch
Start with a simple Hello World example.
Use a blueprint
Build a Lambda application from sample code and configuration presets for common use cases.
Container image
Select a container image to deploy for your function.
Basic information
Function name
Enter a name that describes the purpose of your function.
myFunctionName
Use only letters, numbers, hyphens, or underscores with no spaces.

API Gateway


Connected Users and Streaming Dashboards
Web and Mobile Applications
IoT Devices
Private Applications: VPC and On-Premises
Amazon API Gateway
Create, publish, maintain, monitor, and secure APIs at any scale
API Gateway cache
Amazon CloudWatch
AWS Lambda
Amazon EC2
Amazon Kinesis
Amazon DynamoDB
Other AWS services
Publicly accessible endpoints
Private Applications
VPC and On-Premises
Data Center

API Gateway

- A fully managed service to create, publish, maintain, monitor, and secure APIs at any scale
- The Front Door for applications to access data, business logic, or functionality from backend services
- Restful API: Serverless workloads and HTTP backends
- WebSocket API: To build real-time two-way communications, such as chat apps or streaming dashboards

API Gateway

- Efficient API development
- Performance at any scale
- Cost savings at scale
- Easy monitoring
- Flexible security controls

Rest API

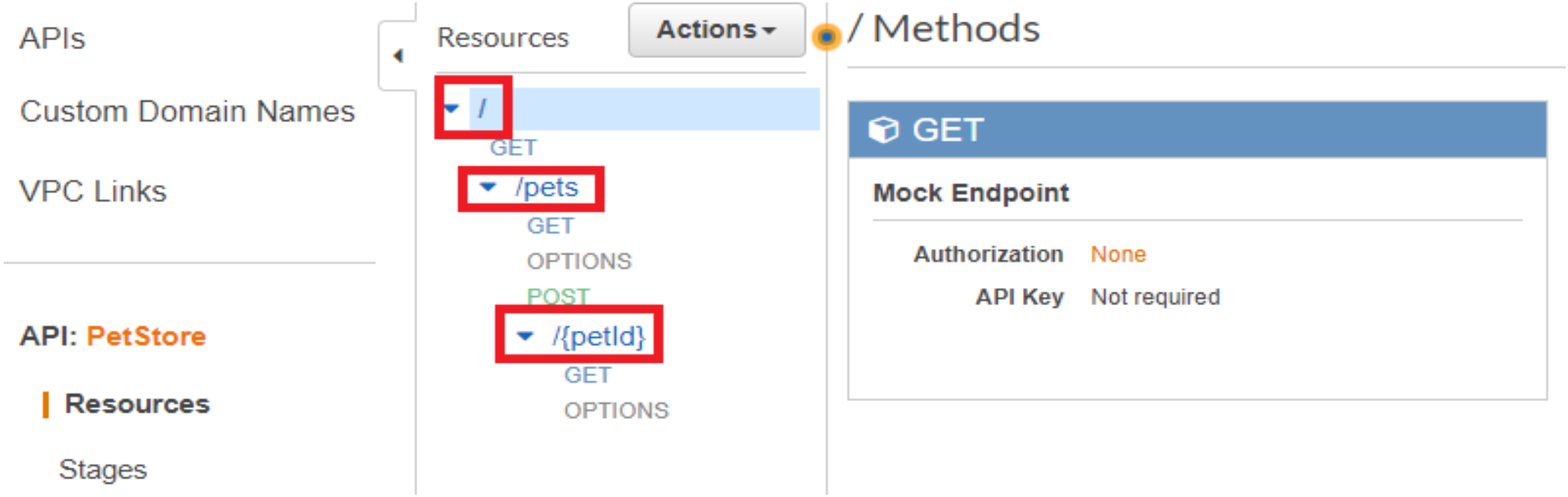
Representational state transfer (REST) is a software architectural style which uses a subset of HTTP.

A Web service that follows these guidelines is called RESTful. Such a Web service must provide its **web resources** in a textual representation and allow them to be read and modified with a **stateless** protocol and a **predefined set of operations**.

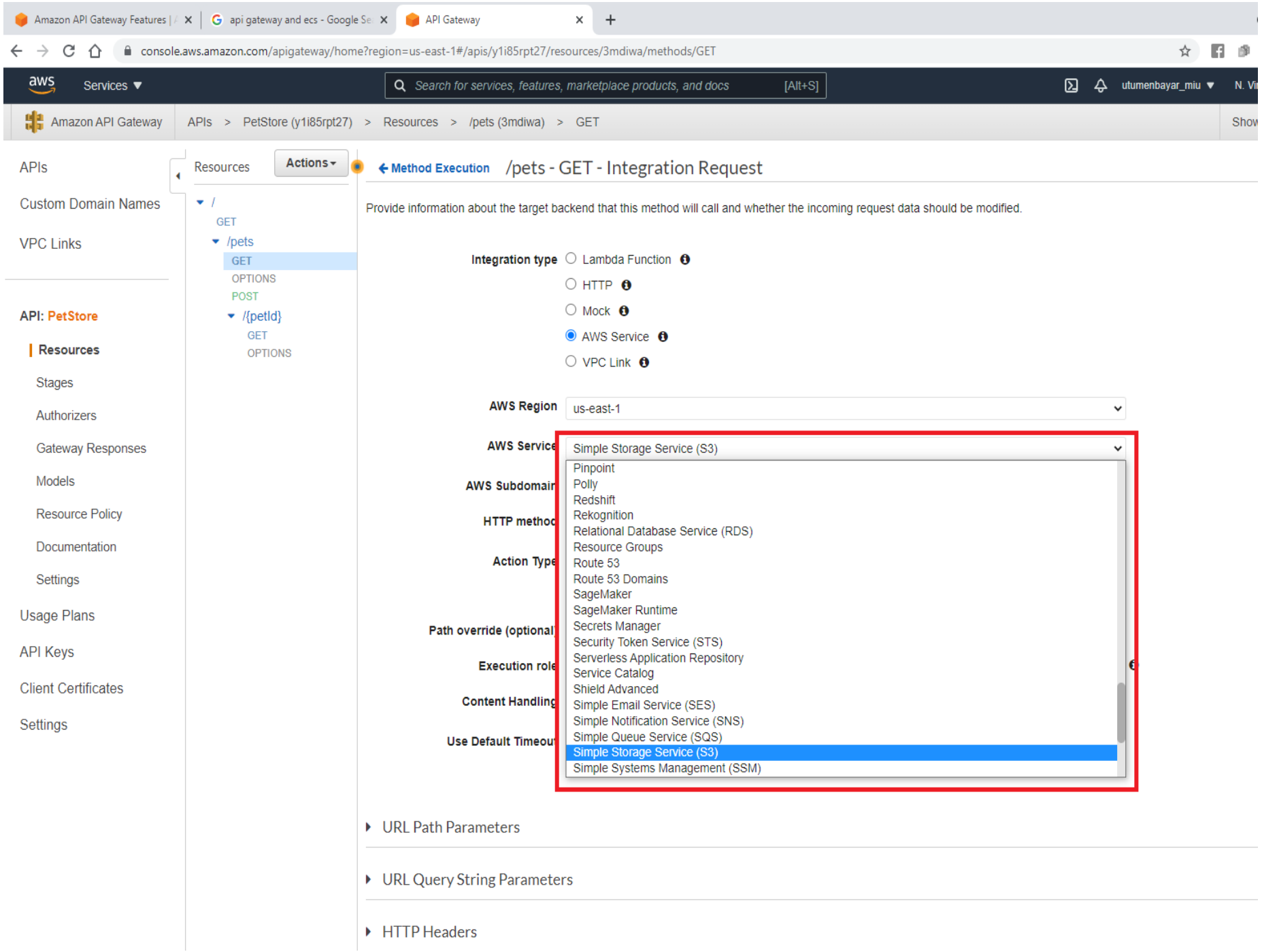
There is a contract between the service provider and consumer

Rest API

Resources are in red. Each resource can have multiple http methods.

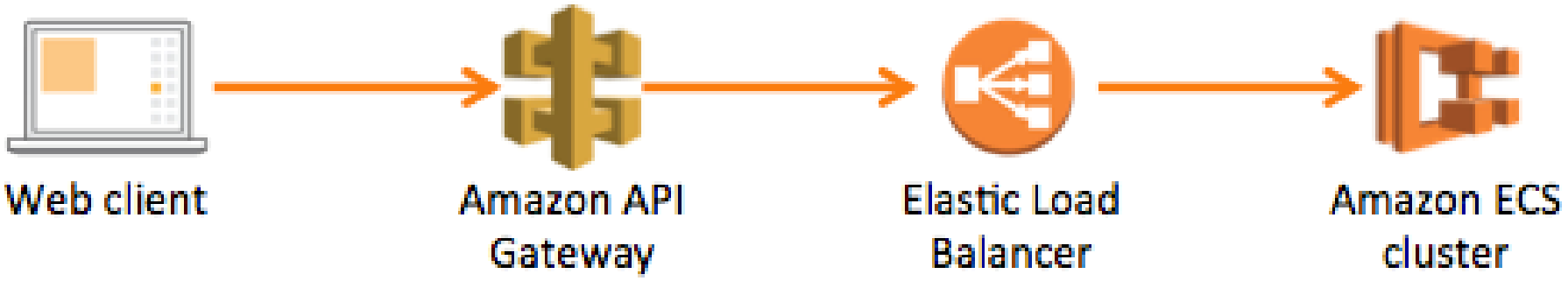


You can run many other AWS services behind API Gateway



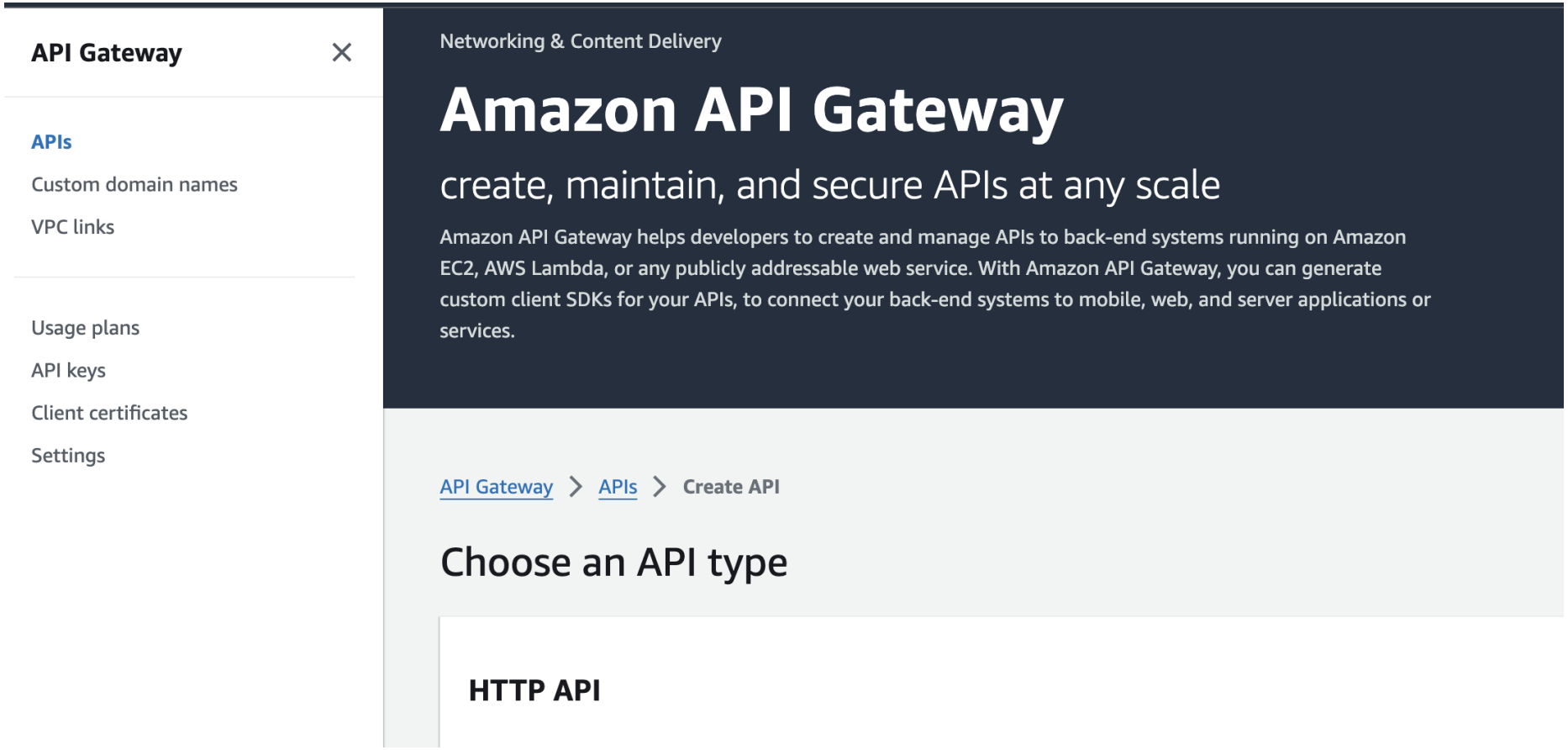
HTTP proxy mode

Behind API gateway, you can run literally run any publicly accessible APIs on on-premise or in the cloud such as ALB, ECS, EC2.



API Gateway

- Demo



Amazon Cognito

- A service lets you add user sign-up, sign-in, and access control to your web and mobile apps quickly and easily.
- Amazon Cognito scales to millions of users and supports sign-in with social identity providers, such as Facebook, Google, and enterprise identity providers, such as Microsoft Active Directory.

Amazon Cognito - Benefits

- **Scalable user directory –** Scales to hundreds of millions of users. No server or infrastructure to manage.
- **Social and enterprise identity federation –** Supports identity and access management standards, such as OAuth 2.0, SAML 2.0.
- **Security for your apps and users –** Multi-factor authentication and encryption of data-at-rest and in-transit.
- **Access control for AWS resources –** You can define roles and map users to different roles so your app can access only the resources that are authorized for each user.
- **Easy integration with your app –** With a built-in UI and easy configuration. You can add your branding. That built-in UI, you can use as a SSO to your internal applications. There is also API Amplify library, with that you can easily integrate your front-end app with Cognito.

Amazon Cognito: Role-based access control

- With Amazon Cognito user pools groups, you can manage your users and their access to resources. It is like a role.
- Use case is, you want to allow only **admin** users **write** access. Whereas other users only have access to read endpoints. That can be achieved with a Cognito group and **custom authorizer.** For more details: Use API Gateway Lambda authorizers
- You can associate groups with IAM roles for more fine-grained access control.

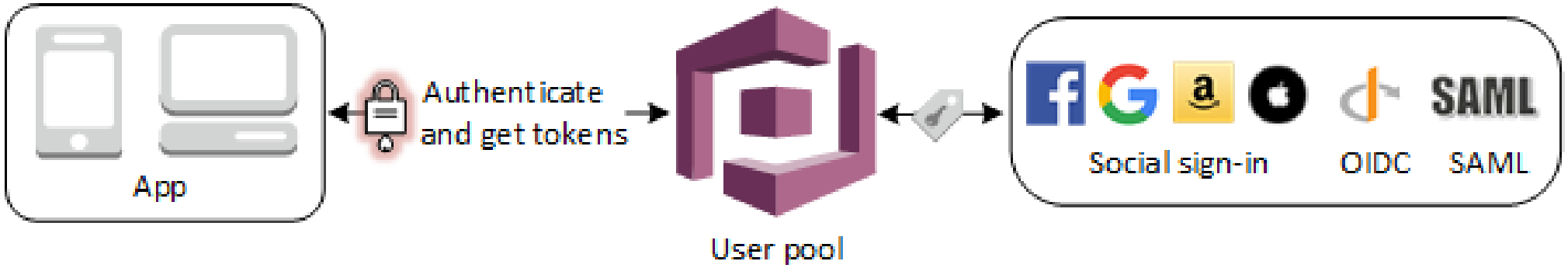
Amazon Cognito - Searchable Attributes

These are searchable attributes in Cognito meaning if you search by these attributes, it is fast. Whereas all other attributes are slow.

- username (case-sensitive)
- email
- phone_number
- name
- given_name
- family_name
- preferred_username
- cognito:user_status (called Status in the Console) (case-insensitive)

Amazon Cognito - JWT token

After successfully authenticating a user, Amazon Cognito issues JSON web tokens (JWT) that you can use to secure and authorize access to your own APIs, or exchange for AWS credentials.



Amazon Cognito - JWT token

A JWT is a structured security token format used to encode JSON data.

Using a JWT allows the token to be validated locally, without making an HTTP request back to the IdP, thereby increasing your application's performance.

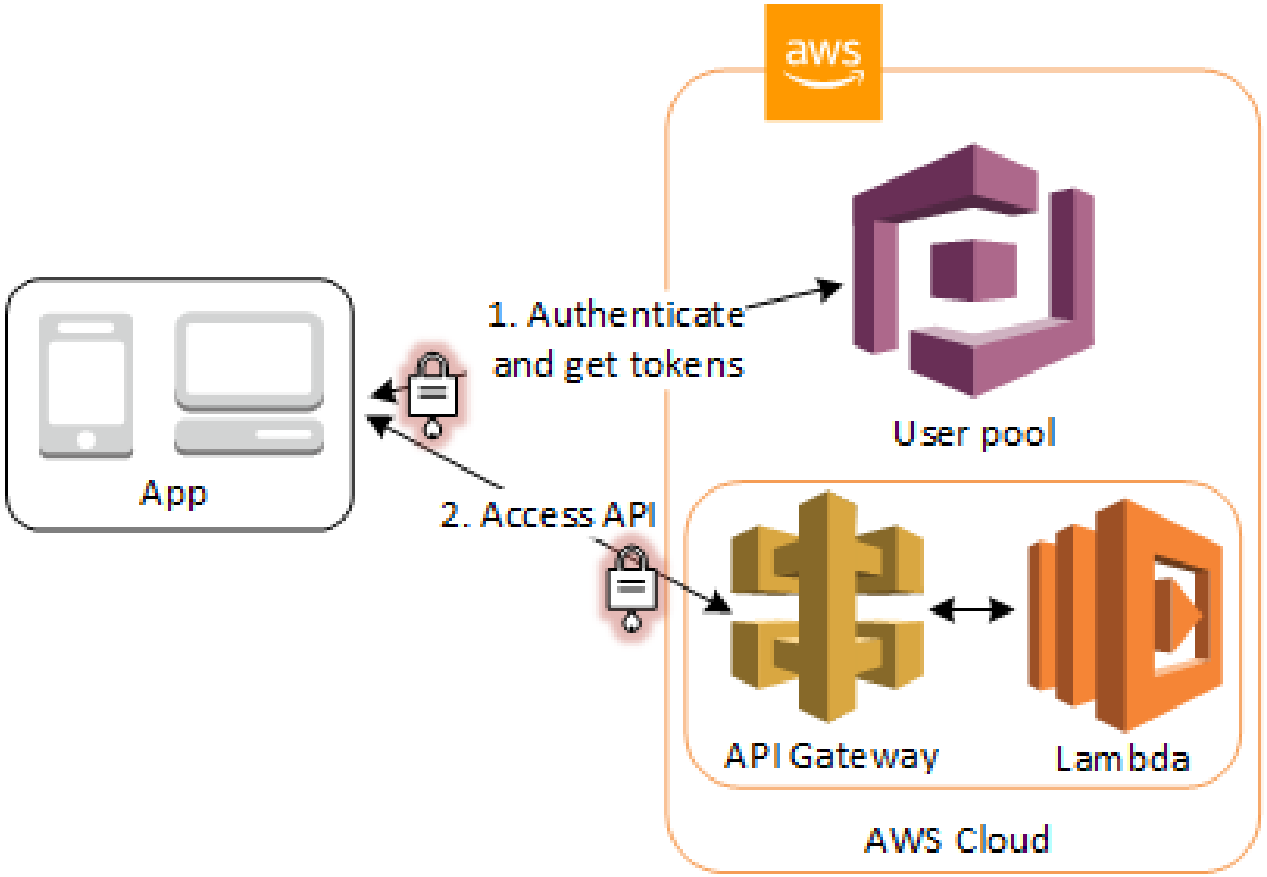
Applications can make use of data inside the token, further reducing expensive HTTP calls and database lookups.

JWT can be stored in a shared caching server so applications can scale out easily as servers don't need to store user session.

Even if the hackers compromised the token, it is **temporary**.

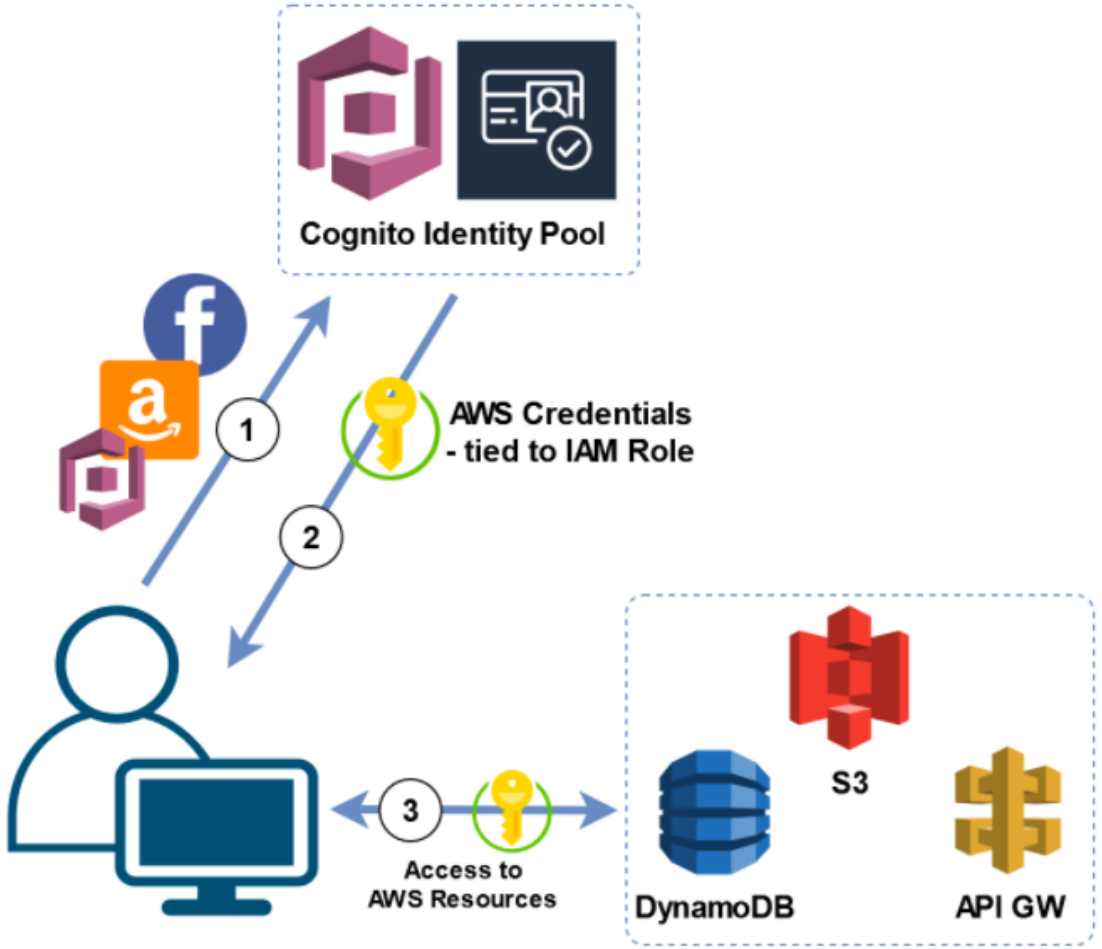
Without identity pool

One token for all users. Then the app has the IAM role that has all permissions. It is a security disadvantage.



With identity pool

The IAM role is tied with the user, not app. Some users will have read only access whereas admins will have all permissions. That is knows as fine-grained access. Better security.



References

- https://docs.aws.amazon.com/
- ChatGPT: https://chatgpt.com/
- Google AI: https://gemini.google.com/app
- https://containersonaws.com/visuals/ecs-architecture/